

Expressions,
opérateurs, structures
de contrôles, fonctions

Les expressions

- Une expression = *"tout ce qui a une valeur"*, et peut donc être affecté à une variable
 - Variable, constante, valeur
 - Appel de fonction
 - Combinaison de variables/constantes/fonctions avec des opérateurs

```
<?php
$a = 12 ;
$b = $a + 12;
$c = $b + strlen("hello world");
```

■ Opérateurs Arithmétiques

Example	Name	Result
$\$a + \b	Addition	Sum of \$a and \$b.
$\$a - \b	Subtraction	Difference of \$a and \$b.
$\$a * \b	Mult.	Product of \$a and \$b.
$\$a / \b	Division	Quotient of \$a and \$b.
$\$a \% \b	Modulus	Remainder of \$a divided by \$b.

■ Opérateurs incrément/décrément

Example	Name	Effect
$++\$a$	Pre_increment	Increments \$a by one, then returns \$a.
$\$a++$	Post_increment	Returns \$a, then increments \$a by one.
$--\$a$	Pre_decrement	Decrements \$a by one, then returns \$a.
$\$a--$	Post_decrement	Returns \$a, then decrements \$a by one.

■ Opérateurs de comparaison

Example	Name	Result
\$a == \$b	Equal	true si \$a est égal à \$b après le transtypage.
\$a === \$b	Identical	true si \$a est égal à \$b et qu'ils sont de même type.
\$a != \$b	Not equal	true si \$a est différent de \$b après le transtypage.
\$a <> \$b	Not equal	true si \$a est différent de \$b après le transtypage.
\$a !== \$b	Not identical	true si \$a est différent de \$b ou ne sont pas du même type
\$a < \$b	Less than	true si \$a est strictement plus petit que \$b.
\$a > \$b	Greater than	true si \$a est strictement plus grand que \$b.
\$a <= \$b	Less than or equal to	true si \$a est plus petit ou égal à \$b.
\$a >= \$b	Greater than or equal to	true si \$a est plus grand ou égal à \$b.

== ou === ?

- == : comparaison **avec** conversion de type

```
12 == 12           : true
```

```
12 == "12", 100 == "1e2"    : true
```

```
"12" == "012" , "10" == "1e1": true (!!!)
```

- === : comparaison **sans** conversion de type

```
12 === 12          : true
```

```
12 === "12" , "12" === "012"      : false
```

- **Recommandation** : utiliser systématiquement === et

!=

— (sauf cas justifié)

Opérateur ternaire `? :` et fusion null `??`

- `(expr1) ? (expr2) : (expr3)`

évaluée **expr2** si `expr1` vaut **true**

évaluée **expr3** si `expr1` vaut **false**

```
$b = 5 - 12 ;  
$a = ($b > 0) ? $b : 0 ;  
  
print $a ;          /* 0 */
```

- `(expr1) ?? (expr2)`

évaluée **expr2** si `expr1` est null ou n'existe pas, sinon **expr1**

```
$c = $val ?? 0 ;  
print $c;           /* 0 */
```

■ Opérateurs Logiques

Example	Name	Result
<code>\$a && \$b</code>	And	TRUE if both \$a and \$b are TRUE .
<code>\$a \$b</code>	Or	TRUE if either \$a or \$b is TRUE .
<code>\$a xor \$b</code>	Xor	TRUE if either \$a or \$b is TRUE , but not both.
<code>! \$a</code>	Not	TRUE if \$a is not TRUE .
<code>\$a and \$b</code>	And	TRUE if both \$a and \$b are TRUE .
<code>\$a or \$b</code>	Or	TRUE if either \$a or \$b is TRUE .

■ Opérateurs sur les chaînes :

Example	Name	Result
<code>\$s1 . \$s2</code>	Concat	The string built by concatenating s1 and s2

Conditionnelles

■ if :

```
if ( $a < 10) {  
    print "$a inférieur a 10" ;  
} else {  
    print "$a supérieur ou égal à 10" ;  
}
```

```
if ( $a < 10) {  
    print "$a inférieur a 10" ;  
} elseif ($a < 20 ) {  
    print "$a inférieur a 20" ;  
} else {  
    print "$a supérieur ou égal à 20" ;  
}
```


■ SWITCH

```
switch ( $a ) {
```

```
  case v1 :
```

```
    <statements1>  
    break;
```

exécuté si <expr> == v1

```
  case v2 :
```

```
    <statements2>  
    break;
```

exécuté si <expr> == v2

```
  ...
```

```
  case vn :
```

```
    <statementsn>  
    break;
```

exécuté si <expr> == vn

```
  default :
```

```
    <statementsd>
```

exécuté dans les autres cas

```
}
```

- match : évalue une expression choisie selon une valeur et retourne le résultat

```
$res = match ( <expr> ) {  
    cond_expr1 => return_exp1,  
    cond_expr2 => return_exp2,  
    cond_expr3 => return_exp3,  
    default    => default_exp  
}
```

évaluée si <expr> === <cond_expr1>
évaluée si <expr> === <cond_expr2>
évaluée si <expr> === <cond_expr3>
évaluée si aucun match

```
$res = match ( $x+1 ) {  
    1 => "un",  
    2 => "deux",  
    $b+4 => "b+4"  
};
```

Attention : si aucun match : erreur fatale

■ La boucle For :

```
for ($i = 1; $i <= 10; $i++) {  
    echo $i;  
}
```

■ La boucle while

```
$i = 1;  
while ($i <= 10) {  
    echo $i++;  
}
```

```
$i = 1;  
do {  
    echo $i++;  
} while ($i <= 10)
```

Les fonctions en php

- Une fonction PHP reçoit des arguments, effectue un calcul et retourne un résultat,
 - le nombre d'arguments peut être variable,
 - le type du résultat peut varier selon les cas d'utilisation de la fonction
- Un appel de fonction est une expression :
 - Affectation à une variable
 - Utilisation dans une expression plus complexe

```
$s = "Oh la belle bleue";  
$l = strlen($s);  
print strlen("hop là")+ $l;
```

Fonctions internes

- PHP dispose de très nombreuses fonctions internes.
- Elles sont complétées par d'autres fonctions qui requièrent que des extensions PHP spécifiques soient installées.
- Documentées dans `php.net` rubrique `manuel/référence des fonctions`
 - Attention : pas facile de chercher, il y en a beaucoup
 - Fonctions sur les chaînes de caractères, les variables, les tableaux, le comportement de l'interprète, les classes et les objets, la manipulation de médias (audio, img, vidéo), les fichiers, les bases de données ...

strlen

(PHP 4, PHP 5, PHP 7, PHP 8)

strlen — Calcule la taille d'une chaîne

versions de php dans lesquelles la fonction existe
rôle de la fonction

Description

```
strlen(string $string): int
```

type attendu de la valeur passée en argument

type de la valeur retournée en résultat

Retourne la taille de la chaîne **string**.

Liste de paramètres

string

La chaîne de caractères à mesurer.

Valeurs de retour

La taille de la chaîne **string** en cas de succès ou 0 si **string** est vide.

Exemples : fonctions sur les variables

```
unset(mixed $var, mixed ...$vars): void
```

unset() détruit la ou les variables dont le nom a été passé en argument **var**.

```
isset(mixed $var, mixed ...$vars): bool
```

Détermine si une variable est considérée définie, ceci signifie qu'elle est déclarée et est différente de **null**.

Si une variable a été détruite avec la fonction [unset\(\)](#), elle n'est plus considéré comme définie.

```
gettype(mixed $value): string
```

Retourne le type de la variable **value**. Pour vérifier le type de la variable, vous pouvez utiliser les fonctions `is_*`.

```
is_null(mixed $value): bool
```

Indique si la variable donnée vaut **null**.

```
is_string(mixed $value): bool
```

Détermine si la variable donnée est de type chaîne de caractères.

Définir des fonctions

■ Syntaxe

```
function name ([type $arg1,type $arg2, ...]): type {  
    <statements>  
    [return ...;]  
}
```

■ Exemple

```
function add (int $a, int $b) : int {  
    return $a + $b ;  
}  
  
$v1 = 24;  
print add($v1, 5);
```


- Déclarations de type des arguments (optionnel, mais obligatoire ;)
 - **Sans mode strict**, PHP tente de transtyper dans le type attendu
 - **Avec mode strict**, si la valeur donnée est d'un type incorrect alors une erreur fatale est générée : une **exception TypeError** est levée.
 - Types possibles :
 - types simples : `bool`, `float`, `int`, `string`...
 - nom de classe/interface, `array`, `callable`
 - `mixed`

```
<?php
declare(strict_types=1);
function test(bool $param):void {}
test(true);
test("true");
```

OK

Fatal error: Uncaught TypeError: test(): Argument #1 (\$param) must be of type bool, string given, [...]

- Déclaration des types de retour (optionnel mais obligatoire)
 - Mêmes types que pour les arguments + `void`
 - **En mode faible (par défaut)** : la valeur de retour est **transtypée**

```
<?php
function sum(float $a, float $b): float {
    return $a + $b;
}
var_dump(sum(1, 2));
```

Retourne
un float

- **En mode strict** : exception **TypeError** si mauvais type

```
<?php
declare(strict_types=1);
function sum2($a, $b): int {
    return $a + $b;
}
var_dump(sum2(1, 2.5));
```

Fatal error:
Uncaught TypeError:
sum(): Return value
must be of type int,
float returned [...]

- Types nullable : les déclarations de types d'arguments et de retour peuvent être indiquées comme nullable en préfixant le nom du type par un ?
- À l'appel, la valeur d'un argument ou du résultat peut alors être **null** au lieu du type attendu sans provoquer d'erreur

```
function bar ( ?string $s ) : ?int {  
    if (is_null($s)) return null;  
    return strlen($s);  
}  
print "\n";  
print bar("yop"); print "\n";  
print bar(null); print "\n";
```

- Types multiples : lorsqu'une fonction peut accepter un même argument de différents type ou retourner des valeurs de différents types
- Deux solutions :
 - Utiliser le type `mixed` : lorsque la valeur peut prendre un type quelconque
 - Utiliser une union de types : lorsque la valeur peut prendre plusieurs types identifiés

```
function  
multitypes( string|bool $v, mixed $l): int|float|null  
{ /*...*/ }
```

■ Valeurs par défaut

- Il est possible de définir une **valeur par défaut** pour certains paramètres
- ils deviennent alors **optionnels** : en cas d'absence à l'appel, le paramètre reçoit la valeur par défaut
- L'ordre est important

```
function foo(int $x,int $y=3,?int $z=null):void
{
    echo "$x\n";
    echo "$y\n";
    echo is_null($z) ? "null" : "not null";
}
```

1
3
null

4
5
null

```
foo(1); // y et z oubliés : valeurs par défaut
foo(4,5); // c'est forcément z qui est oublié
foo(4,5,6);
```

4
5
not null

■ arguments nommés

- Il est possible de nommer les arguments lors de l'appel d'une fonction en s'appuyant sur le nom du paramètre
- Permet plus de souplesse dans l'ordre et les valeurs par défaut

```
function foo(int $x,int $y=3,?int $z=null):void {  
    echo "$x\n";  
    echo "$y\n";  
    if (is_null($z)) { echo "null" ; }  
    else echo "not null";  
};  
  
foo(y: 0, x: 1 );  
foo(x:1, z:4);  
foo(12, z: 6);
```

1
4
null

1
3
not null

12
3
not null

Inclusion de code

■ **require()** :

- inclut et évalue le code contenu dans le fichier spécifié

■ **require_once()**

Identique à **require()**, sauf que le fichier n'est inclus qu'une seule fois

```
<?php  
  
require 'prepend.php';  
  
require $somefile;  
  
require ('somefile.txt');  
  
require_once 'autoload.php';
```